Maps in JavaScript

Introduction

A Map is a data structure that stores data in key-value pairs. Unlike objects, Maps can use any data type as a key (including objects, functions, or primitives), and they maintain the insertion order of elements.

Creating a Map

There are two ways to create a Map in JavaScript:

1. Empty Map

```
const map = new Map();
```

Output:

Map(0) {}

2. Map with Initial Values

You can initialize a Map with an array of key-value pairs:

```
const userRoles = new Map([
    ["Ali", "admin"],
    ["Nitin", "user"],
    ["Vinod", "superuser"]
]);
```

Output:

Powered by SPRK Technologies

```
Map(3) { 'Ali' => 'admin', 'Nitin' => 'user', 'Vinod' => 'superuser' }
```

Basic Map Operations

Adding Elements: set()

The `set()` method adds a new key-value pair to the Map.

```
userRoles.set("Aditi", "admin");
```

Output:

```
Map(4) { 'Ali' => 'admin', 'Nitin' => 'user', 'Vinod' => 'superuser', 'Aditi' => 'admin' }
```

Updating Elements: set()

The same `set()` method updates an existing key's value if the key already exists.

```
userRoles.set("Nitin", "superuser");
```

Output:

```
Map(4) { 'Ali' => 'admin', 'Nitin' => 'superuser', 'Vinod' => 'superuser', 'Aditi' => 'admin' }
```

Retrieving Values: get()

The `get()` method returns the value associated with a specific key.

```
console.log(userRoles.get("Nitin"));
```

Output:

superuser

Checking Key Existence: has()

The `has()` method returns `true` if the key exists in the Map, otherwise `false`.

```
console.log(userRoles.has("Vinod"));
```

Output:

true

Deleting Elements: delete()

The `delete()` method removes a key-value pair from the Map.

```
userRoles.delete("Aditi");
console.log(userRoles.size);
```

Output:

3

Getting Map Size: size

The `size` property returns the number of key-value pairs in the Map.

```
console.log(userRoles.size);
```


Powered by SPRK Technologies

Output:

3

Iterating Over Maps

1. Iterating Over Keys and Values

Use the `for...of` loop to iterate over both keys and values simultaneously.

```
for(const [key, value] of userRoles){
    console.log(`${key}: ${value}`);
}
```

Output:

Ali: admin Nitin: superuser Vinod: superuser

2. Iterating Over Keys Only

The `keys()` method returns an iterator containing all the keys.

```
for(const key of userRoles.keys()){
    console.log(`${key}`);
}
```

Output:

Ali Nitin Vinod

3. Iterating Over Values Only

The `values()` method returns an iterator containing all the values.

Powered by SPRK Technologies

```
for(const value of userRoles.values()){
    console.log(`${value}`);
}
```

Output:

admin superuser superuser

4. Iterating Over Entries

The `entries()` method returns an iterator of [key, value] pairs.

```
for(const entry of userRoles.entries()){
    console.log(entry);
}
```

Output:

['Ali', 'admin'] ['Nitin', 'superuser'] ['Vinod', 'superuser']

Converting Between Data Types

1. Array to Map

You can convert an array of key-value pairs into a Map.

```
const arr = [["name", "Amit"], ["age", 30]];
const arrMap = new Map(arr);
console.log(arrMap);
```

Output:

Powered by SPRK Technologies

Map(2) { 'name' => 'Amit', 'age' => 30 }

2. Object to Map

To convert an object to a Map, use `Object.entries()` which returns an array of key-value pairs.

```
const person = {name: "Aditi", age: 34};
const mapObj = new Map(Object.entries(person));
console.log(mapObj);
```

Output:

Map(2) { 'name' => 'Aditi', 'age' => 34 }

3. Map to Object

To convert a Map back to an object, use `Object.fromEntries()`.

```
const rolesObj = Object.fromEntries(userRoles);
console.log(rolesObj);
```

Output:

{ Ali: 'admin', Nitin: 'superuser', Vinod: 'superuser' }

4. Map to Array

Use the spread operator (`...`) to convert Map keys, values, or entries into arrays.

```
const allKeys = [...userRoles.keys()];
console.log(allKeys);
```

Output:

['Ali', 'Nitin', 'Vinod']

The same technique works for values and entries:

```
const allValues = [...userRoles.values()];
const allEntries = [...userRoles.entries()];
```

Output:

['admin', 'superuser', 'superuser'] [['Ali', 'admin'], ['Nitin', 'superuser'], ['Vinod', 'superuser']]

When to Use Maps

Maps are useful when:

- You need to use non-string keys (objects, functions, etc.)
- You need to maintain the insertion order of elements
- You frequently add or remove key-value pairs
- You need to check the size of the collection frequently

Map vs Object

While both Maps and Objects store key-value pairs, there are important differences:

- **Key Types:** Map keys can be any data type, while object keys are always strings or symbols
- **Order:** Maps maintain insertion order, objects do not guarantee order
- **Size:** Maps have a built-in `size` property, objects require manual counting
- **Performance:** Maps perform better for frequent additions and deletions

---

End of Document